Solutions for Practice Problems for Week 2

Operating and Systems Programming 2022/23

For all questions involving code, try to answer them *without* running the code first.

1. Local variables are valid only within the block where they are defined and become invalid once the block has finished executing. Heap and global variables are valid throughout the program. Static variables are valid only within the block where they are defined but keep their value between executions of the same block. The compiler allocates space for local, global and static variables, whereas space for heap variables must be allocated via malloc.

2. An example is

   ```
   void foo (int *a, int n) {
      int i;

      for (i = 0; i < n; i++) {
        printf ("a[%d] = %d\n", i, a[i]);
      }
   }
   ```

   Calling the function `foo` creates a stack frame which contains the variables `a, n` and `i`. These variables are inaccessible after the function `foo` has been executed.

3. Because the variable which the pointer points to is inaccessible after the function has returned, the effects are unpredictable, eg return of a random value or a program crash. An example would be

   ```
   int *foo() {
      int i = 4;

      return &i;
   }
   ```

4. When pass-by-value is used, the value of the argument is copied into a local variable. Hence any modification of the local variable has no

effect outside the function. When pass-by-reference is used, a pointer is passed as an argument of a function. Hence any modification of the value referenced by the pointer is visible outside the function as well.

5. `temp` is a local variable of the function `max`, hence is no longer accessible once this function has been executed. Therefore the pointer to `temp` which is returned as the result of this function, points to an invalid location, and unpredictable effects will result.

6.
```
#include <stdio.h>
#include <stdlib.h>
int main () {
    int i, j;
    int *p[4];

    for (i = 0; i < 4; i++) {
if ((p[i] = malloc(4*sizeof(int))) == NULL) {
    fprintf (stderr, "Cannot allocate memory, exiting\n");
    exit (1);
}
for (j = 0; j < 4; j++) {
    p[i][j] = 4*i + j;
}
    }

    return 0;
}
```

7. `foo1` returns a pointer to a local variable. Hence the pointers points to an illegal address after the function exits. `foo2` does not allocate any memory for for `p`, hence `*p` may override an arbitrary location in memory.

8. The memory allocated in line 3 is released only when the program finishes. Hence this is technically a memory leak but it is OK in this case, as the program terminates when `p` is no longer used.